LLM-Based Verilog Adder Generation, Verification, and Yosys-Guided PPA Optimization

Huancheng Yang

April 17, 2026

Abstract

This report documents an end-to-end workflow for LLM-assisted digital design using two 8-bit adders as case studies: a Ripple Carry Adder (RCA8) and a Carry Lookahead Adder (CLA8). The project is divided into three stages. In Part 1, golden Verilog designs were reverse-engineered into natural-language architectural descriptions and then regenerated using an LLM with architecture-preserving prompts. In Part 2, self-checking testbenches with internal-signal verification were generated and used with Icarus Verilog to validate both functionality and internal behavior. In Part 3, the Yosys synthesis framework was connected to an LLM in an iterative optimization loop to explore power-performance-area (PPA) tradeoffs under area, delay, balanced, and multi-start settings. The final results show that the LLM could reproduce and verify correct adder implementations, and that the synthesis-guided loop could discover useful tradeoff points; however, PPA improvement was only meaningful when functional correctness and equivalence were preserved.

1. Introduction

Large Language Models (LLMs) are increasingly useful for structured code generation, but hardware design requires stronger validation than ordinary software generation. A generated RTL block must preserve interface correctness, architecture intent, synthesis compatibility, simulation behavior, and, when optimization is involved, equivalence to the baseline design. This project studies that workflow in three connected phases:

1. regenerate Verilog from architectural descriptions,
2. generate and validate self-checking testbenches with internal-signal checking, and
3. close a feedback loop between an LLM and Yosys to optimize PPA.

The selected adders were **RCA8** and **CLA8**. RCA8 was chosen because it is structurally simple and makes carry propagation easy to inspect. CLA8 was chosen because it introduces generate/propagate logic and explicit lookahead carry equations, providing a good contrast with RCA8 and testing whether an LLM can preserve a more structured architecture.

2. Part 1: LLM-Based Verilog Generation from Golden Designs

2.1 Selected Adders and Rationale

The two selected designs were:

- **RCA8**: chosen as the simplest 8-bit adder architecture and as a baseline for understanding stage-by-stage carry propagation.
- **CLA8**: chosen to study a faster architecture based on propagate/generate signals and explicit carry-lookahead logic.

This pairing covers both ends of the complexity spectrum: a structurally simple ripple-carry design and a more optimized arithmetic structure.

2.2 Natural-Language Descriptions

For both designs, the golden Verilog was first translated into architecture-preserving natural-language specifications. The descriptions were written not as loose summaries, but as regeneration-oriented specifications that preserved:

- exact module names,
- top-level ports and ordering,
- helper-module interfaces,
- important internal signals,
- module hierarchy and instantiation style,
- implementation style constraints.

For RCA8, the description emphasized the `FA` helper module, the carry chain `c[7:1]`, and the ripple-carry organization from LSB to MSB. For CLA8, the description explicitly preserved the `PGGen` helper module, internal vectors `g`, `p`, `c`, and representative `e` terms used in carry-lookahead equations.

2.3 LLM-Generated Verilog

The regenerated RTL was produced with prompts that strongly prioritized architecture fidelity over brevity. The prompts explicitly required preservation of:

- module names,
- port names, directions, widths, and ordering,
- important internal signal declarations,
- helper-module usage,

- structural design style.

This prompt design was essential. A looser prompt often produced functionally plausible but architecturally inconsistent RTL, especially for CLA8. Tightening the prompt greatly improved the structural similarity between generated code and the golden reference.

2.4 Manual Verification

The generated designs were manually reviewed against the golden designs using the following checklist:

- module names,
- port declarations,
- internal signals,
- module hierarchy,
- implementation style,
- sub-module instantiation consistency.

RCA8 showed strong architectural agreement with the golden design. The generated version preserved the `FA`+`RCA8` hierarchy and the ripple-carry structure. CLA8 required more prompt refinement, but the final version preserved the key `PGGen`+`CLA8` organization and the intended carry-lookahead structure closely enough to proceed to dynamic verification in Part 2.

3. Part 2: LLM-Based Testbench Generation and Simulation

3.1 Input Preparation and Internal Signals

The inputs to the testbench-generation stage consisted of the architecture-preserving natural-language descriptions from Part 1 and the regenerated RTL for both adders. Critical internal signals were identified as follows:

- **RCA8**: carry chain `c[1]` through `c[7]`.
- **CLA8**: `cin`, propagate and generate vectors `p[7:0]` and `g[7:0]`, carry vector `c[0:7]`, and representative lookahead terms such as `e[0]`, `e[3]`, `e[10]`, `e[21]`, and `e[35]`.

These internal signals were explicitly listed in the prompts so that the generated testbenches would not only verify outputs but also confirm that the architecture's internal behavior was correct.

3.2 Generated Testbenches

The testbenches were written as self-checking Verilog environments with the following features:

- DUT instantiation using the exact module interface,

- directed and randomized test vectors,
- expected-value computation,
- explicit checking of internal signals,
- `pass_count`, `fail_count`, and `total_tests` counters,
- waveform dumping via `$dumpfile` and `$dumpvars`,
- final `PASS/FAIL` summary.

During development, several testbench issues had to be debugged, including missing expected-value declarations, incomplete internal-signal checks, and syntax problems in generated Verilog. These were corrected by either refining the prompt or replacing incomplete generated fragments with manually repaired self-checking code.

3.3 Simulation Results

Both final testbenches passed simulation in Icarus Verilog. Representative commands were:

```
iverilog -g2012 -o rca8_sim RCA8_llm_for_tb.v RCA8_tb.v
vvp rca8_sim

iverilog -g2012 -o cla8_sim CLA8_llm_for_tb.v CLA8_tb.v
vvp cla8_sim
```

The simulation logs showed the following results:

- **RCA8**: all 60 test cases passed; the carry chain behaved correctly stage by stage.
- **CLA8**: all 60 test cases passed after testbench correction; the generate/propagate logic, carry vector, and representative lookahead intermediates matched the expected values.

3.4 Part 2 Analysis

The final analysis established that:

- all tested primary outputs passed,
- internal signals matched the expected architecture-specific behavior,
- RCA8 carry propagation was correct through `c[1:7]`,
- CLA8 carry behavior matched the intended lookahead equations rather than ripple propagation,
- no unexpected signal values remained after debugging.

Thus, Part 2 provided functional evidence that the regenerated RCA8 and CLA8 designs were correct for the tested cases and that their internal signal behavior matched the intended architecture.

4. Part 3: Yosys-Based PPA Optimization Loop

4.1 Yosys Setup and PPA Extraction

Yosys was installed in Colab and used together with the Nangate 45nm liberty file. A reusable synthesis flow was built around:

- `synth_adder.ys`,
- `run_yosys.py`,
- `constraints.sdc`.

A practical limitation of the Colab environment was that the provided Yosys/ABC version did not reliably report `logic_levels`. In addition, ABC issued warnings that the SDC commands such as `create_clock`, `set_input_delay`, and `set_output_delay` were not fully recognized in this environment. Therefore, the optimization experiments used `delay_ps` extracted from the ABC timing report as the practical delay metric.

For the baseline RCA8 design, the synthesized PPA numbers were:

- area = 53.2 μm^2,
- cell count = 40,
- delay = 74.83 ps.

4.2 Single-Start Optimization Modes from RCA8

The RCA8 baseline was optimized under three modes:

1. **Area mode**
2. **Delay mode**
3. **Balanced mode**

The best results are summarized in Table 1.

Table 1: RCA8 optimization results across the three modes.

Mode	Cell Count	Area (μm^2)	Delay (ps)
Area	40	53.2	74.83
Delay	90	129.808	70.84
Balanced	40	61.712	37.54

The interpretation of these results is as follows:

- **Area mode**: the best result remained effectively the original RCA8 structure. This suggests the baseline RCA8 was already very competitive for minimum cell count under this mapping flow.

- **Delay mode**: delay improved, but at a large area cost. More importantly, this mode later failed functional verification and had to be rejected.
- **Balanced mode**: the best balanced design preserved the same cell count as baseline while reducing delay substantially, at the cost of somewhat larger mapped area.

4.3 Architecture Identification

The best designs were interpreted with an LLM to understand the structural tendencies induced by the optimization loop. The balanced RCA8 design was interpreted as a **Hybrid RCA/CLA** structure, which is consistent with the observed PPA pattern: delay improved significantly while area rose moderately, suggesting that the synthesis flow chose a faster mapped implementation without reducing logical complexity in the same proportion.

4.4 Verification of Optimized Designs

Each best design was then re-verified using a dedicated functional testbench. The verification outcome is summarized in Table 2.

Table 2: Verification status of the best designs from the single-start RCA8 optimization loop.

Mode	Functional Simulation	SAT Equivalence	Final Decision
Area	PASS	PASS	Accepted
Delay	FAIL	FAIL / not required	Rejected
Balanced	PASS	PASS	Accepted

The key conclusion from Step 7 is that **PPA improvement cannot be accepted without preserving functional correctness**. The delay-oriented candidate improved delay metrics but failed functional simulation, so it was rejected. In contrast, the area-mode and balanced-mode candidates passed both simulation and SAT-based equivalence checking against the RCA8 baseline and were therefore accepted.

4.5 Multi-Start Exploration

To avoid local optima, the optimization loop was also run from multiple starting architectures. The final comparison table is shown in Table 3.

Table 3: Multi-start exploration results. Logic levels are marked N/A because the current Colab/Yosys environment did not report them reliably; delay in ps was used instead.

Start Arch.	Mode	Final Cells	Logic Levels	Area (μm^2)	Delay (ps)
KSA8	Area	32	N/A	44.688	131.25
CLA8	Delay	38	N/A	50.274	224.01
RCA8	Balanced	37	N/A	67.564	201.71

The identified architecture labels for these runs were:

- **KSA8 + area** → Prefix / Kogge-Stone-like,
- **CLA8 + delay** → Ripple Carry,
- **RCA8 + balanced** → Hybrid RCA/CLA.

This result is important because it shows that the LLM-Yosys loop did not simply preserve the starting architecture. Instead, depending on the optimization objective and starting point, it could converge toward very different design families.

5. Discussion

5.1 Accuracy and Limitations of LLM-Generated RTL

This project showed that LLM-generated RTL can be made reliable when the prompts are sufficiently explicit and when simulation is used as a hard correctness filter. However, the process also exposed several limitations:

- weak prompts tend to produce architecturally inconsistent code,
- testbench generation may omit declarations or expected-value logic,
- synthesis-guided optimization can produce candidates with attractive PPA but incorrect functionality,
- different tool versions may require workflow adaptation.

In other words, the LLM was useful as a design assistant, but not as a design oracle.

5.2 Quality of the Generated Testbenches

The generated testbenches became effective only after explicitly requesting internal-signal verification and self-checking behavior. For arithmetic structures such as CLA8, internal signals mattered greatly. Output-only checking would not have revealed whether carry-lookahead logic was implemented in the intended way.

5.3 Lessons Learned and Best Practices

Several practical lessons emerged from this project:

1. **Preserve architecture explicitly in the prompt.** Exact module names, hierarchy, signal declarations, and implementation style should be named as hard constraints.
2. **Treat simulation as mandatory.** A design that synthesizes is not necessarily correct.
3. **Separate "tool ran" from "design passed."** In Part 3, the delay-mode optimized design compiled and simulated, but still failed the functional testbench.
4. **Expect environment-specific adaptation.** In Colab with Yosys 0.9, `logic_levels` and SDC handling required practical workarounds, so `delay_ps` was used instead.

5. **Formal checking is worth the effort.** SAT-based equivalence checking was necessary to distinguish a true optimized implementation from a merely different one.

6. Conclusion

This project successfully completed the full LLM-assisted hardware workflow required by the assignment: architecture interpretation, Verilog regeneration, self-checking testbench generation, RTL simulation, Yosys-driven PPA optimization, and final verification. RCA8 and CLA8 both reached functionally verified regenerated implementations. In the Yosys optimization loop, the **area-mode** and **balanced-mode** results were accepted after functional simulation and SAT-based equivalence checking, while the **delay-mode** result was rejected because its improved delay came at the cost of incorrect behavior.

The final outcome shows both the promise and the limits of LLM-guided digital design. LLMs can accelerate structured RTL generation and architecture exploration, but only when paired with strict prompts, simulation-based validation, and formal checks. The most valuable lesson from this project is not simply that LLMs can generate hardware, but that they can be integrated into a disciplined verification-driven workflow that keeps correctness as the non-negotiable requirement.

Appendix: Deliverables Checklist

The following project outputs were produced or documented in the notebook workflow:

- adder selection and justification,
- natural-language descriptions for RCA8 and CLA8,
- LLM-generated Verilog for both adders,
- Part 1 verification notes,
- internal-signal-checking testbenches,
- Iverilog simulation outputs,
- Part 2 analysis and verification summaries,
- Yosys scripts and configuration files,
- optimization logs and PPA summaries,
- best optimized designs and Step 7 verification results,
- multi-start comparison table,
- architecture-identification summaries,
- this final report.